

Remote Development Project Report

Student Name: Dai Kaixuan
Student ID: [ZY2557202]

1. Project Overview

This project practices command line operations, Markdown documentation, and matrix multiplication implementation. The main task is to implement matrix multiplication in Python, verify the correctness of the algorithm, and document the process in a Markdown report.

2. System Configuration

Since this project was developed on macOS, I used macOS command line tools to collect system information.

Item	Command Used	Result
CPU Model	<code>sysctl -n machdep.cpu.brand_string</code>	Apple M1
Memory Size	<code>system_profiler SPHardwareDataType grep "Memory"</code>	8 GB
Operating System	<code>sw_vers</code>	macOS 14.7.4
Kernel Version	<code>uname -a</code>	Darwin 23.6.0
Compiler Version	<code>gcc --version</code>	Apple clang version 16.0.0
Python Version	<code>python --version</code>	Python 3.9.13
## 3. Python Language Implementation		

The Python implementation is stored in `assignment1/matrix_mul.py`.

The program implements matrix multiplication using the standard triple-loop algorithm. If matrix A has shape $m \times n$ and matrix B has shape $n \times p$, then the output matrix C has shape $m \times p$.

For each element $C[i][j]$, the algorithm multiplies the i -th row of matrix A with the j -th column of matrix B, and then sums the products.

The mathematical expression is:

$$C[i][j] = \sum(A[i][k] * B[k][j])$$

The core part of the Python implementation is:

```
for i in range(m):
    for j in range(p):
        total = 0
        for k in range(n):
            total += A[i][k] * B[k][j]
        C[i][j] = total
```

This implementation does not use built-in matrix multiplication functions for the main algorithm. NumPy is only used later to verify the correctness of the result.

Execution Command

The Python program can be executed with:

```
python assignment1/matrix_mul.py --size 3
```

The `--size` argument controls the size of the square matrices used for NumPy verification and benchmarking.

4. Algorithm Verification

The correctness of the matrix multiplication algorithm was verified in two ways.

First, I used a small 2×2 matrix example. The input matrices were:

```
A = [[1, 2],  
     [3, 4]]
```

```
B = [[5, 6],  
     [7, 8]]
```

According to the definition of matrix multiplication, the expected result is:

```
[[19, 22],  
 [43, 50]]
```

The program output was:

```
Small matrix verification:  
A = [[1, 2], [3, 4]]  
B = [[5, 6], [7, 8]]  
Result = [[19, 22], [43, 50]]  
Expected = [[19, 22], [43, 50]]  
Correct: True
```

This shows that the implementation correctly handles a simple manually verified case.

Second, I compared my implementation with NumPy matrix multiplication. NumPy was only used as a reference result, not as the main implementation.

For matrix size 3 x 3, the output was:

```
NumPy verification:  
Matrix size: 3 x 3  
Correct: True
```

Since the result of my implementation matched NumPy's result, the algorithm passed the NumPy-based correctness check.

5. Python Benchmark

The program also measures the execution time of the pure Python implementation.

For matrix size 3 x 3, the benchmark result was:

```
Python benchmark:  
Matrix size: 3 x 3  
Execution time: 5.708999999964881e-06 seconds
```

This is a very small test case, so the execution time is extremely short. Larger matrix sizes are more useful for observing performance differences between different programming languages or implementations.

6. C Language Implementation

A C version of the matrix multiplication program was also implemented in `assignment1/matrix_mul.c`.

The C implementation uses the same standard triple-loop algorithm as the Python version. The main difference is that the C program stores matrices in one-dimensional arrays. For a matrix with size $n \times n$, the element at row i and column j is accessed as:

```
M[i * n + j]
```

The core multiplication logic is:

```
for (int i = 0; i < n; i++) {  
    for (int j = 0; j < n; j++) {  
        int total = 0;  
        for (int k = 0; k < n; k++) {  
            total += A[i * n + k] * B[k * n + j];  
        }  
        C[i * n + j] = total;  
    }  
}
```

The C program can be compiled with:

```
gcc -O2 assignment1/matrix_mul.c -o assignment1/matrix_mul_c
```

It can be executed with:

```
./assignment1/matrix_mul_c 3
```

The C implementation was verified using the same 2 x 2 test case:

```
Small matrix verification:  
Result = [[19, 22], [43, 50]]  
Expected = [[19, 22], [43, 50]]  
Correct: True
```

This confirms that the C implementation produces the correct result for the manually verified test case.

7. Conclusion

In this assignment, I practiced basic command line operations, collected system configuration information, implemented matrix multiplication in Python, and verified the correctness of the algorithm using both a manually checked example and NumPy comparison.

I also implemented a C version of the same algorithm. This helped me understand the difference between an interpreted language implementation and a compiled language implementation at a basic level. Both implementations follow the same mathematical definition of matrix multiplication.

Through this project, I became more familiar with Markdown documentation, terminal commands, Git version control, and the process of preparing technical work for publication on a static website.

8. References

- Python Documentation
- NumPy Documentation
- GCC / Clang command line documentation
- macOS command line tools
- Course assignment instructions

9. Appendix

The main files created for this assignment are:

```
assignment1/matrix_mul.py  
assignment1/matrix_mul.c  
assignment1/report.md  
assignment1/results/system_info.txt  
assignment1/results/python_test_size3.txt  
assignment1/results/c_test_size3.txt
```

The generated executable file `assignment1/matrix_mul_c` is ignored by Git because it can be rebuilt from the C source code.